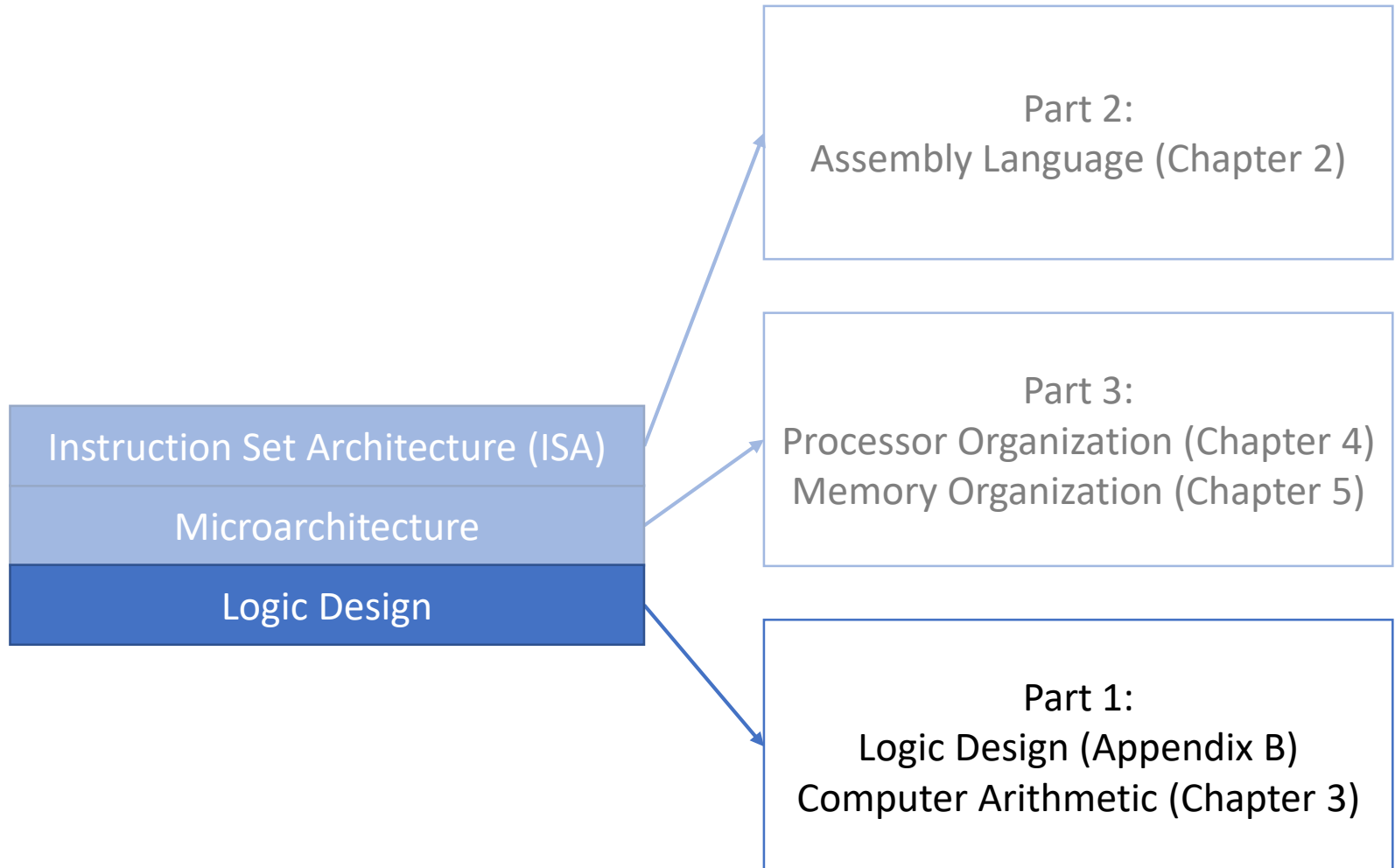


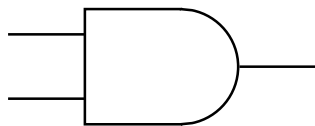
Lecture 4: Logic Design Fundamentals

CMPS 221 – Computer Organization and Design

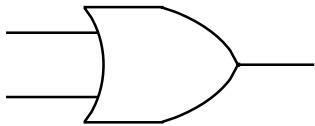
Part 1: Logic Design



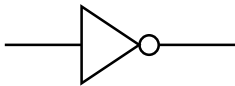
Logic Design



AND

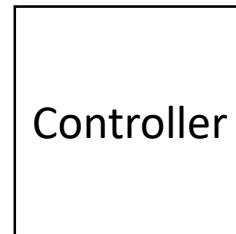
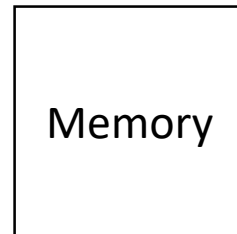
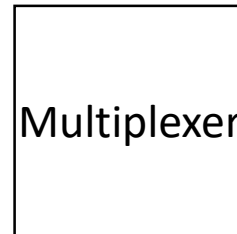
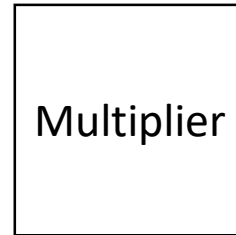
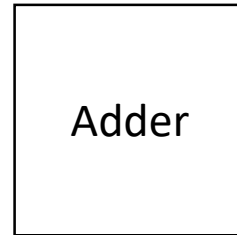
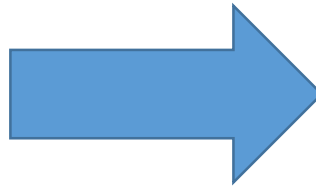


OR



NOT

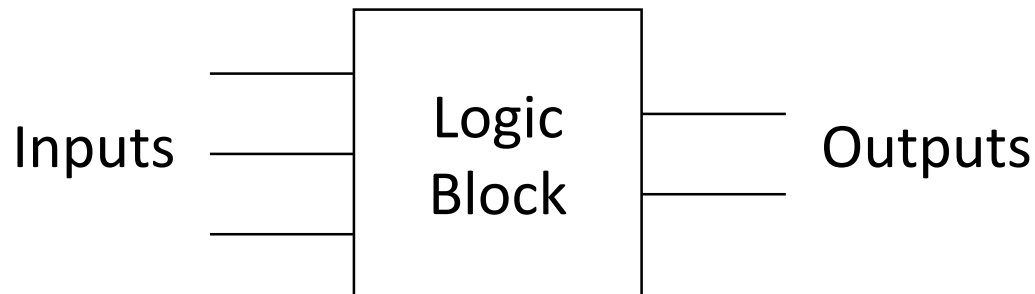
Logic Gates



Logic Blocks

Logic Blocks

- Computers are built from circuits that store charge and process electric signals in one of two states
 - High voltage (hereon, denoted as 1)
 - Low voltage (hereon, denoted as 0)
- A **logic block** is a circuit that performs operations on a set of input signals (0's and 1's) to produce a set of output signals (0's and 1's)



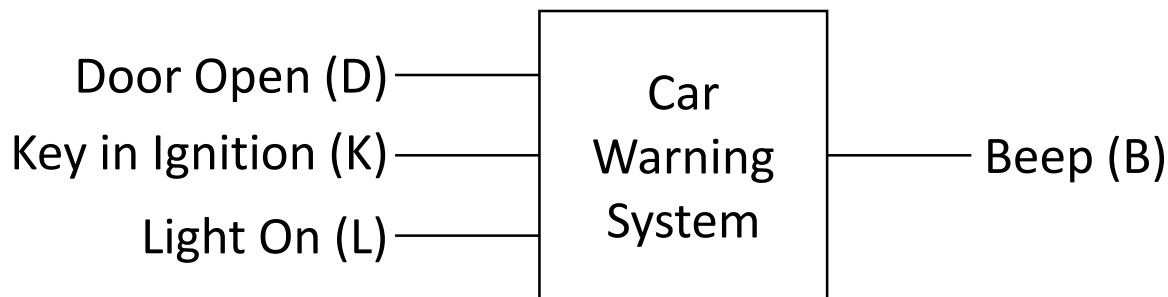
Types of Logic Blocks

- The two general categories of logic blocks are:
 - **Combinational Logic**: output depends only on the current input
 - **Sequential Logic**: output depends on both the current input as well as the values stored in the block (memory)
- } focus of today

Truth Tables

- One way of specifying the behavior of a logic block is using **truth tables** which exhaustively list what the output should be for every input combination

- Example:



Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Logic Equations

- Another way of specifying the behavior of a logic block is using **logic equations** based on Boolean algebra
- In Boolean algebra:
 - All variables are either 0 or 1
 - Typical formulations have three fundamental operators:
 - OR (+): $A + B$ is 1 if either A or B (or both) are 1
 - AND ($\cdot$): $A \cdot B$ is 1 if both A and B are 1
 - NOT ($\bar{}$): $\bar{A}$ is 1 if A is 0

Logic Equations Example

Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$B = \bar{D} \cdot \bar{K} \cdot L + D \cdot \bar{K} \cdot L + D \cdot K \cdot \bar{L} + D \cdot K \cdot L$$

Boolean Algebra Identities

- Identity Laws:

$$A + 0 = A$$

$$A \cdot 1 = A$$

- Zero/One Laws:

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

- Inverse Laws:

$$A + \bar{A} = 1$$

$$A \cdot \bar{A} = 0$$

Boolean Algebra Identities

- Commutative Laws:

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

- Associative Laws:

$$A + (B + C) = (A + B) + C$$

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

- Distributive Laws:

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Boolean Algebra Identities

- DeMorgan's Laws:

$$\overline{A + B + \cdots + C} = \bar{A} \cdot \bar{B} \cdot \cdots \cdot \bar{C}$$

$$\overline{A \cdot B \cdot \cdots \cdot C} = \bar{A} + \bar{B} + \cdots + \bar{C}$$

Applying Identities

Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$B = \bar{D} \cdot \bar{K} \cdot L + \textcolor{blue}{D} \cdot \bar{K} \cdot L + \textcolor{blue}{D} \cdot K \cdot \bar{L} + \textcolor{blue}{D} \cdot K \cdot L$$

$$B = \bar{D} \cdot \bar{K} \cdot L + \textcolor{blue}{D} \cdot (\bar{K} \cdot L + \textcolor{green}{K} \cdot \bar{L} + \textcolor{green}{K} \cdot L) \quad \text{(Distributive)}$$

$$B = \bar{D} \cdot \bar{K} \cdot L + \textcolor{blue}{D} \cdot (\bar{K} \cdot L + \textcolor{green}{K} \cdot (\bar{L} + L)) \quad \text{(Distributive)}$$

$$B = \bar{D} \cdot \bar{K} \cdot L + \textcolor{blue}{D} \cdot (\bar{K} \cdot L + \textcolor{green}{K} \cdot \textcolor{brown}{1}) \quad \text{(Inverse)}$$

$$B = \bar{D} \cdot \bar{K} \cdot L + \textcolor{blue}{D} \cdot (\bar{K} \cdot L + \textcolor{green}{K}) \quad \text{(Identity)}$$

Applying Identities

Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$B = \bar{D} \cdot \bar{K} \cdot L + D \cdot \bar{K} \cdot L + D \cdot K \cdot \bar{L} + D \cdot K \cdot L$$

$$B = (\bar{D} + D) \cdot \bar{K} \cdot L + D \cdot K \cdot \bar{L} + D \cdot K \cdot L$$

$$B = \bar{K} \cdot L + D \cdot K \cdot \bar{L} + D \cdot K \cdot L$$

$$B = \bar{K} \cdot L + D \cdot K \cdot (\bar{L} + L)$$

$$B = \bar{K} \cdot L + D \cdot K$$

(Distributive)

(Inverse)

(Distributive)

(Inverse)

Uniqueness of Specification

truth table is unique

Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$B = \bar{D} \cdot \bar{K} \cdot L + D \cdot \bar{K} \cdot L + D \cdot K \cdot \bar{L} + D \cdot K \cdot L$$

logic equations are not unique

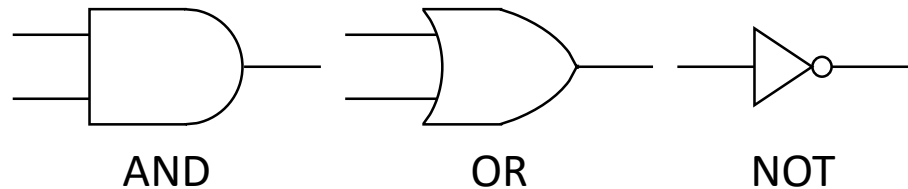
(many possible equations to
specify the same function)

$$B = \bar{D} \cdot \bar{K} \cdot L + D \cdot (\bar{K} \cdot L + K)$$

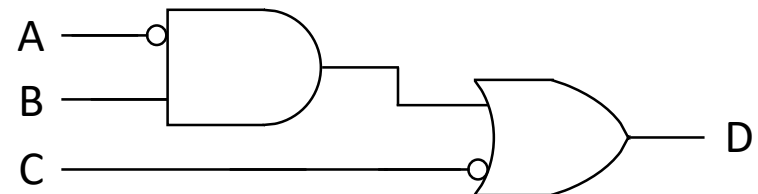
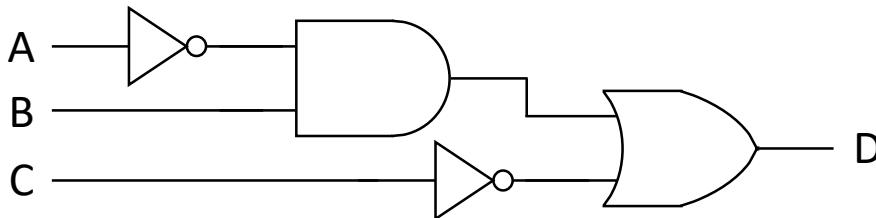
$$B = \bar{K} \cdot L + D \cdot K$$

Logic Gates

- Logic blocks are built from **logic gates** which are circuits that implement basic logic functions
- Any logic function can be implemented using three basic types of gates:



- Example logic diagram: $D = \bar{A} \cdot B + \bar{C}$

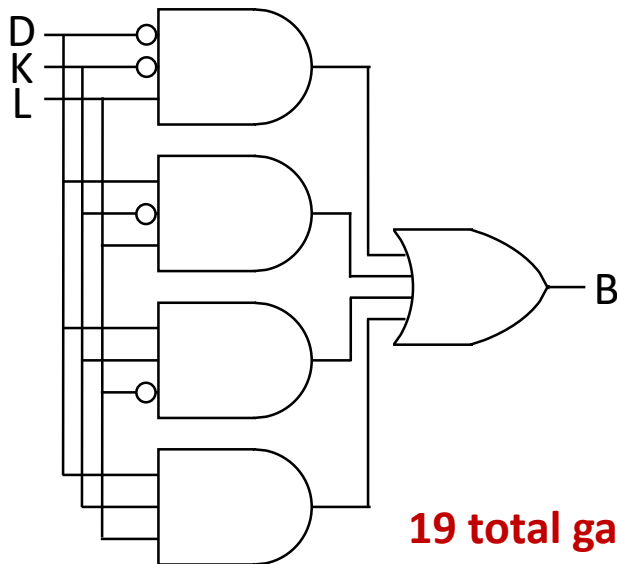


Using "bubbles" as shorthand for inverters

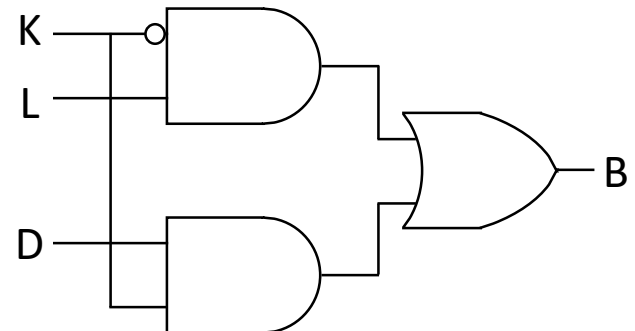
Uniqueness of Implementation

- Like logic equations, logic diagrams are not unique, which gives flexibility in implementing logic blocks
 - Challenge: finding the most efficient implementation (uses the smallest *number of gate inputs*)

$$B = \bar{D} \cdot \bar{K} \cdot L + D \cdot \bar{K} \cdot L + D \cdot K \cdot \bar{L} + D \cdot K \cdot L$$



$$B = \bar{K} \cdot L + D \cdot K$$



19 total gate inputs

7 total gate inputs

Karnaugh Maps

- A **Karnaugh Map** (K-map) is a technique for simplifying Boolean algebra expressions so they can be implemented with a small number of gates and inputs per gate
- General approach:
 - Layout truth table on a two-dimensional grid
 - Search for largest rectangles with a size that is a power of two to cover all the 1's

Two Variable K-map Example

Inputs		Output
A	B	C
0	0	1
0	1	1
1	0	0
1	1	1

$$C = \bar{A} \cdot \bar{B} + \bar{A} \cdot B + A \cdot B$$

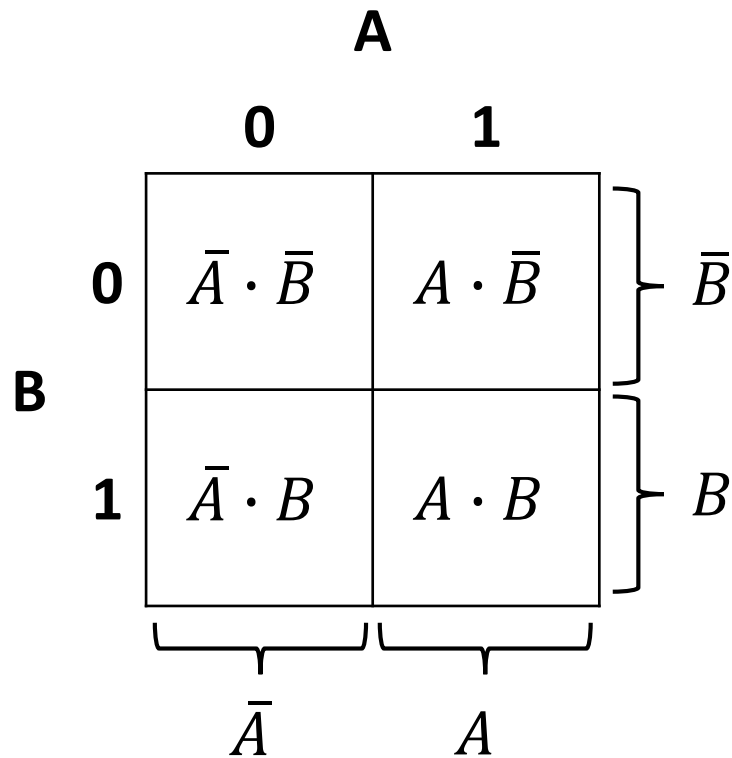
Truth Table

		A	
		0	1
B	0	1	0
	1	1	1

$$C = \bar{A} + B$$

K-map

Two Variable K-map



Three Variable K-map

		AB			
		00	01	11	10
C	0	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	$\bar{A} \cdot B \cdot \bar{C}$	$A \cdot B \cdot \bar{C}$	$A \cdot \bar{B} \cdot \bar{C}$
	1	$\bar{A} \cdot \bar{B} \cdot C$	$\bar{A} \cdot B \cdot C$	$A \cdot B \cdot C$	$A \cdot \bar{B} \cdot C$

Diagram illustrating the Three Variable K-map grid. The grid is a 2x4 table with columns labeled AB (00, 01, 11, 10) and rows labeled C (0, 1). The cells contain the minterm expressions. Brackets below the grid indicate the grouping of cells for variables A and B. A bracket on the right side of the grid indicates the grouping of cells for variable C.

Use gray code to order the cells on the grid. **Gray code** is an ordering of binary numbers where successive numbers differ by just one bit.

Three Variable K-map Example

Inputs			Output
A	B	C	D
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Truth Table

		AB			
		00	01	11	10
C	0	0	1	1	1
	1	0	1	1	0

Diagram illustrating the K-map with groupings for variables B and A.

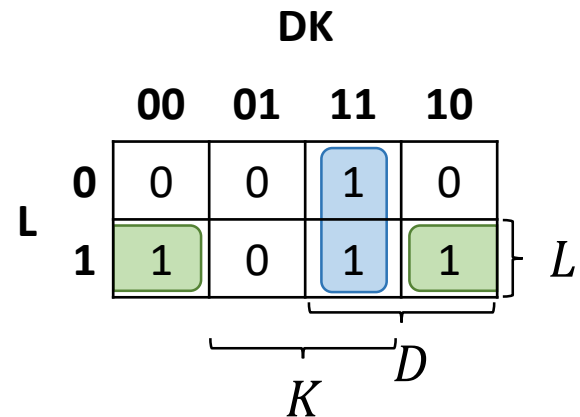
$$D = B + A \cdot \bar{C}$$

K-map

Car Warning System Example

Inputs			Output
D	K	L	B
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Truth Table



$$B = \bar{K} \cdot L + D \cdot K$$

K-map

Textbook Sections

- Some of the content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - B.1, B.2